

# Data Summarization

Data Wrangling in R

# Data Summarization

- Basic statistical summarization
  - `mean(x)`: takes the mean of x
  - `sd(x)`: takes the standard deviation of x
  - `median(x)`: takes the median of x
  - `quantile(x)`: displays sample quantiles of x. Default is min, IQR, max
  - `range(x)`: displays the range. Same as `c(min(x), max(x))`
  - `sum(x)`: sum of x
  - `max(x)`: maximum value in x
  - `min(x)`: minimum value in x
- all have the `na.rm = argument` for missing data

# Statistical summarization

These functions work on **vectors**:

```
x <- c(1, 5, 7, 4, 2, 8)  
mean(x)
```

```
[1] 4.5
```

Summarization on a `data.frame/tibble`:

```
mtcars %>% pull(hp) %>% mean()
```

```
[1] 146.6875
```

## GUT CHECK!

What kind of object do we need to run summary operators like `mean()` ?

- A. A vector of numbers
- B. A vector of characters
- C. A dataset

# UFO Data

Let's use the UFO data again:

```
ufo <- read_csv("https://sisbid.github.io/Data-Wrangling/data/ufo/ufo_data_complete.csv")
ufo <- ufo %>% rename(duration_s = `duration (seconds)`)
head(ufo)
```

```
# A tibble: 6 × 11
  datetime city state country shape duration_s `duration (hours/min)` comments
  <chr>    <chr> <chr> <chr>   <chr>      <dbl> <chr>                        <chr>
1 10/10/19... san ... tx    us    cyli...    2700 45 minutes                This ev...
2 10/10/19... lack... tx    <NA>    light    7200 1-2 hrs                  1949 La...
3 10/10/19... ches... <NA>    gb     circ...     20 20 seconds                Green/O...
4 10/10/19... edna  tx    us     circ...     20 1/2 hour                  My olde...
5 10/10/19... kane... hi    us     light     900 15 minutes                AS a Ma...
6 10/10/19... bris... tn    us     sphe...     300 5 minutes                My fath...
# i 3 more variables: `date posted` <chr>, latitude <chr>, longitude <chr>
```

# Column to vector

Summarize a column as a vector with `pull()`.

```
ufo %>% pull(duration_s) %>% mean()
```

```
[1] NA
```

```
ufo %>% pull(duration_s) %>% mean(na.rm=TRUE)
```

```
[1] 8373.412
```

```
ufo %>% pull(duration_s) %>% median(na.rm=TRUE)
```

```
[1] 120
```

## dplyr: count

**count**: the number of rows in each group.

```
ufo %>% count(country)
```

```
# A tibble: 6 × 2  
  country      n  
  <chr>    <int>  
1 au         593  
2 ca        3266  
3 de         112  
4 gb        2050  
5 us       70293  
6 <NA>    12561
```

## dplyr: count

Multiple columns listed further subdivides the count.

```
ufo %>% count(country, shape)
```

```
# A tibble: 139 × 3
```

|    | country | shape    | n     |
|----|---------|----------|-------|
|    | <chr>   | <chr>    | <int> |
| 1  | au      | changing | 10    |
| 2  | au      | chevron  | 3     |
| 3  | au      | cigar    | 18    |
| 4  | au      | circle   | 68    |
| 5  | au      | cone     | 8     |
| 6  | au      | cross    | 3     |
| 7  | au      | cylinder | 10    |
| 8  | au      | diamond  | 11    |
| 9  | au      | disk     | 62    |
| 10 | au      | egg      | 12    |

```
# i 129 more rows
```

## dplyr: count

Option to sort the results with `sort = TRUE`

```
ufo %>% count(country, sort = TRUE)
```

```
# A tibble: 6 × 2
  country      n
  <chr>    <int>
1 us      70293
2 <NA>    12561
3 ca       3266
4 gb       2050
5 au        593
6 de       112
```

## GUT CHECK!

The `count()` function can help us tally:

- A. Sample size
- B. Rows per each category
- C. How many categories

## dp~~l~~yr: count

Instead of counting the **number of rows** in each group, `wt` computes `sum()` of a column for each group.

```
# Add up "duration_s" for each "country" category  
ufo %>% count(country, wt = duration_s)
```

```
# A tibble: 6 × 2  
  country      n  
  <chr>      <dbl>  
1 au      2047880.  
2 ca      86578311.  
3 de       2546878  
4 gb     125846817.  
5 us     377662105.  
6 <NA>    149463090.
```

Grouping

# Perform Operations By Groups: dplyr

Regular data...

```
# No group_by()
ufo
```

```
# A tibble: 88,875 × 11
```

|    | datetime   | city    | state | country | shape   | duration_s | `duration (hours/min)` | comments   |
|----|------------|---------|-------|---------|---------|------------|------------------------|------------|
|    | <chr>      | <chr>   | <chr> | <chr>   | <chr>   | <dbl>      | <chr>                  | <chr>      |
| 1  | 10/10/1... | san ... | tx    | us      | cyl...  | 2700       | 45 minutes             | This ev... |
| 2  | 10/10/1... | lack... | tx    | <NA>    | light   | 7200       | 1-2 hrs                | 1949 La... |
| 3  | 10/10/1... | ches... | <NA>  | gb      | circ... | 20         | 20 seconds             | Green/O... |
| 4  | 10/10/1... | edna    | tx    | us      | circ... | 20         | 1/2 hour               | My olde... |
| 5  | 10/10/1... | kane... | hi    | us      | light   | 900        | 15 minutes             | AS a Ma... |
| 6  | 10/10/1... | bris... | tn    | us      | sphe... | 300        | 5 minutes              | My fath... |
| 7  | 10/10/1... | pena... | <NA>  | gb      | circ... | 180        | about 3 mins           | penarth... |
| 8  | 10/10/1... | norw... | ct    | us      | disk    | 1200       | 20 minutes             | A brigh... |
| 9  | 10/10/1... | pell... | al    | us      | disk    | 180        | 3 minutes              | Strobe ... |
| 10 | 10/10/1... | live... | fl    | us      | disk    | 120        | several minutes        | Saucer ... |

```
# i 88,865 more rows
```

```
# i 3 more variables: `date posted` <chr>, latitude <chr>, longitude <chr>
```

# Perform Operations By Groups: dplyr

`group_by` allows you group the data set by variables/columns you specify:

```
ufo_grouped <- ufo %>% group_by(country)
ufo_grouped
```

```
# A tibble: 88,875 × 11
# Groups:   country [6]
  datetime city state country shape duration_s `duration (hours/min)` comments
  <chr>    <chr> <chr> <chr>    <chr>    <dbl> <chr>                      <chr>
1 10/10/1... san ... tx us cyli... 2700 45 minutes This ev...
2 10/10/1... lack... tx <NA> light 7200 1-2 hrs 1949 La...
3 10/10/1... ches... <NA> gb circ... 20 20 seconds Green/O...
4 10/10/1... edna tx us circ... 20 1/2 hour My olde...
5 10/10/1... kane... hi us light 900 15 minutes AS a Ma...
6 10/10/1... bris... tn us sphe... 300 5 minutes My fath...
7 10/10/1... pena... <NA> gb circ... 180 about 3 mins penarth...
8 10/10/1... norw... ct us disk 1200 20 minutes A brigh...
9 10/10/1... pell... al us disk 180 3 minutes Strobe ...
10 10/10/1... live... fl us disk 120 several minutes Saucer ...
# i 88,865 more rows
# i 3 more variables: `date posted` <chr>, latitude <chr>, longitude <chr>
```

# Summarize the grouped data

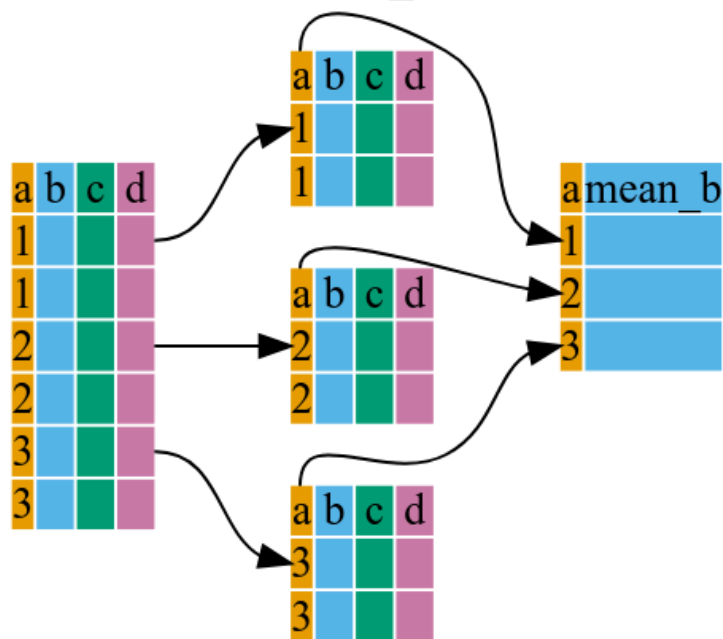
We can summarize grouped data using `summarize()`:

```
ufo_grouped %>%  
  summarize(mean_value = mean(duration_s, na.rm = TRUE))
```

```
# A tibble: 6 × 2  
  country mean_value  
  <chr>      <dbl>  
1 au         3453.  
2 ca        26509.  
3 de        22740.  
4 gb        61389.  
5 us         5373.  
6 <NA>      11901.
```

# Visual of `summarize`

```
gapminder %>%  
group_by(a) %>%  
summarize(mean_b=mean(b))
```



[source](#)

## Use the **pipe** to string these together!

Pipe `ufo` into `group_by`, then pipe that into `summarize`:

```
ufo %>%  
  group_by(country) %>%  
  summarize(mean_value = mean(duration_s, na.rm = TRUE),  
            max_value = max(duration_s, na.rm = TRUE))
```

```
# A tibble: 6 × 3  
  country mean_value max_value  
  <chr>      <dbl>      <dbl>  
1 au         3453.    1209600  
2 ca        26509.   82800000  
3 de        22740.   1814400  
4 gb        61389.   97836000  
5 us         5373.   66276000  
6 <NA>      11901.   52623200
```

## group\_by with mutate - Useful for comparisons

Use `group_by` to calculate the mean value for each country. We can use `mutate` to add it as a column.

```
ufo_compare <- ufo %>%  
  group_by(country) %>%  
  mutate(duration_country_avg = mean(duration_s, na.rm = TRUE)) %>%  
  select(country, duration_s, duration_country_avg)  
ufo_compare
```

```
# A tibble: 88,875 × 3  
# Groups:   country [6]  
  country duration_s duration_country_avg  
  <chr>      <dbl>          <dbl>  
1 us         2700           5373.  
2 <NA>       7200          11901.  
3 gb         20           61389.  
4 us         20           5373.  
5 us         900           5373.  
6 us         300           5373.  
7 gb         180           61389.  
8 us        1200           5373.  
9 us         180           5373.  
10 us        120           5373.  
# i 88,865 more rows
```

## group\_by with mutate - Useful for comparisons

Create a “difference” variable:

```
ufo_compare %>% mutate(diff = duration_s - duration_country_avg)
```

```
# A tibble: 88,875 × 4
```

```
# Groups:   country [6]
```

|    | country | duration_s | duration_country_avg | diff    |
|----|---------|------------|----------------------|---------|
|    | <chr>   | <dbl>      | <dbl>                | <dbl>   |
| 1  | us      | 2700       | 5373.                | -2673.  |
| 2  | <NA>    | 7200       | 11901.               | -4701.  |
| 3  | gb      | 20         | 61389.               | -61369. |
| 4  | us      | 20         | 5373.                | -5353.  |
| 5  | us      | 900        | 5373.                | -4473.  |
| 6  | us      | 300        | 5373.                | -5073.  |
| 7  | gb      | 180        | 61389.               | -61209. |
| 8  | us      | 1200       | 5373.                | -4173.  |
| 9  | us      | 180        | 5373.                | -5193.  |
| 10 | us      | 120        | 5373.                | -5253.  |

```
# i 88,865 more rows
```

Iterative summaries

## Iterative summaries: `dplyr` `summarize()` and `across()` functions

Use the `across` function with `summarize()` to summarize across multiple columns of your data.

```
dropouts <- read_delim("https://sisbid.github.io/Data-Wrangling/data/dropouts.txt", delim = "/")
dropouts %>%
  group_by(ETHNIC) %>%
  summarize(across(c(D9, D10, D11, D12), ~sum(.x)))
```

# A tibble: 9 × 5

|   | ETHNIC | D9    | D10   | D11   | D12   |
|---|--------|-------|-------|-------|-------|
|   | <dbl>  | <dbl> | <dbl> | <dbl> | <dbl> |
| 1 | 0      | 1226  | 299   | 308   | 478   |
| 2 | 1      | 103   | 82    | 124   | 265   |
| 3 | 2      | 118   | 114   | 132   | 767   |
| 4 | 3      | 37    | 25    | 34    | 135   |
| 5 | 4      | 34    | 30    | 51    | 252   |
| 6 | 5      | 4433  | 3543  | 4723  | 16220 |
| 7 | 6      | 817   | 617   | 875   | 2890  |
| 8 | 7      | 768   | 878   | 1432  | 3998  |
| 9 | 9      | 162   | 155   | 194   | 636   |

## Select different columns based on the data class

Use ?tidyr\_tidy\_select functions like `where(is.numeric)`!

```
dropouts %>%  
  group_by(ETHNIC) %>%  
  summarize(across( where(is.numeric), ~ sum(.x, na.rm = TRUE)))
```

# A tibble: 9 × 18

|   | ETHNIC | E7     | E8     | E9     | E10    | E11    | E12    | EUS   | ETOT    | D7    | D8    |
|---|--------|--------|--------|--------|--------|--------|--------|-------|---------|-------|-------|
|   | <dbl>  | <dbl>  | <dbl>  | <dbl>  | <dbl>  | <dbl>  | <dbl>  | <dbl> | <dbl>   | <dbl> | <dbl> |
| 1 | 0      | 2922   | 2709   | 3140   | 3291   | 2776   | 2345   | 43    | 11595   | 66    | 52    |
| 2 | 1      | 2523   | 2589   | 2808   | 2886   | 2889   | 3020   | 33    | 11636   | 16    | 15    |
| 3 | 2      | 44199  | 43898  | 43343  | 43757  | 45840  | 43001  | 596   | 176537  | 69    | 37    |
| 4 | 3      | 2390   | 2284   | 2436   | 2352   | 2471   | 2571   | 42    | 9872    | 9     | 7     |
| 5 | 4      | 12210  | 12883  | 13429  | 14037  | 14273  | 14469  | 233   | 56441   | 12    | 8     |
| 6 | 5      | 254745 | 252583 | 264302 | 260201 | 252458 | 253193 | 2680  | 1032834 | 682   | 550   |
| 7 | 6      | 26383  | 26786  | 28497  | 28731  | 28696  | 30799  | 485   | 117208  | 217   | 164   |
| 8 | 7      | 113498 | 115139 | 115593 | 117592 | 119308 | 121771 | 1829  | 476093  | 323   | 267   |
| 9 | 9      | 14704  | 13469  | 13999  | 13238  | 12810  | 13000  | 138   | 53185   | 59    | 38    |

# i 7 more variables: D9 <dbl>, D10 <dbl>, D11 <dbl>, D12 <dbl>, DUS <dbl>,  
# DTOT <dbl>, YEAR <dbl>

## summary() Function

Using `summary()` can give you rough snapshots of each numeric column (character columns are skipped):

```
summary(dropouts)
```

| CDS_CODE   |          | ETHNIC   |          | GENDER     |           | E7       |            |
|------------|----------|----------|----------|------------|-----------|----------|------------|
| Length     | :59599   | Min.     | :0.000   | Length     | :59599    | Min.     | : 0.000    |
| N.unique   | : 5507   | 1st Qu.: | :2.000   | N.unique   | : 2       | 1st Qu.: | : 0.000    |
| N.blank    | : 0      | Median   | :5.000   | N.blank    | : 0       | Median   | : 0.000    |
| Min.nchar: | 14       | Mean     | :4.737   | Min.nchar: | 1         | Mean     | : 7.946    |
| Max.nchar: | 14       | 3rd Qu.: | :7.000   | Max.nchar: | 1         | 3rd Qu.: | : 3.000    |
|            |          | Max.     | :9.000   |            |           | Max.     | :373.000   |
| E8         |          | E9       |          | E10        |           | E11      |            |
| Min.       | : 0.000  | Min.     | : 0.00   | Min.       | : 0.000   | Min.     | : 0.000    |
| 1st Qu.:   | : 0.000  | 1st Qu.: | : 0.00   | 1st Qu.:   | : 0.000   | 1st Qu.: | : 0.000    |
| Median     | : 0.000  | Median   | : 0.00   | Median     | : 0.000   | Median   | : 0.000    |
| Mean       | : 7.925  | Mean     | : 8.18   | Mean       | : 8.156   | Mean     | : 8.079    |
| 3rd Qu.:   | : 3.000  | 3rd Qu.: | : 2.00   | 3rd Qu.:   | : 2.000   | 3rd Qu.: | : 2.000    |
| Max.       | :363.000 | Max.     | :567.00  | Max.       | :537.000  | Max.     | :620.000   |
| E12        |          | EUS      |          | ETOT       |           | D7       |            |
| Min.       | : 0.000  | Min.     | : 0.000  | Min.       | : 0.00    | Min.     | : 0.000000 |
| 1st Qu.:   | : 0.000  | 1st Qu.: | : 0.000  | 1st Qu.:   | : 0.00    | 1st Qu.: | : 0.000000 |
| Median     | : 0.000  | Median   | : 0.000  | Median     | : 1.00    | Median   | : 0.000000 |
| Mean       | : 8.124  | Mean     | : 0.102  | Mean       | : 32.64   | Mean     | : 0.02438  |
| 3rd Qu.:   | : 2.000  | 3rd Qu.: | : 0.000  | 3rd Qu.:   | : 9.00    | 3rd Qu.: | : 0.000000 |
| Max.       | :490.000 | Max.     | :77.000  | Max.       | :2127.00  | Max.     | :15.000000 |
| D8         |          | D9       |          | D10        |           | D11      |            |
| Min.       | :0.00000 | Min.     | : 0.0000 | Min.       | : 0.00000 | Min.     | : 0.0000   |

# Summary

- summary stats (`mean()`) work with `pull()`
- `count(x)`: what unique values do you have?
- `group_by()`: changes all subsequent functions
  - combine with `summarize()` to get statistics per group
  - combine with `across()` to select several columns
- `summary(x)`: quantile information

[https://sisbid.github.io/Data-Wrangling/08\\_Data\\_Summarization/lab/data-summarization-lab.Rmd](https://sisbid.github.io/Data-Wrangling/08_Data_Summarization/lab/data-summarization-lab.Rmd)